A

Project Report on

JAVASCRIPT TESTING

Submitted for partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

Sreyash Pattnaik - 24K81A05R7

Under the Guidance of

Mr. M. Dileep Kumar

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ST.MARTIN'S ENGINEERING COLLEGE

UGC AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

St. MARTIN'S ENGINEERING COLLEGE

UGC AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

CERTIFICATE

This is to certify that the project titled **"JAVASCRIPT TESTING"** is being submitted by **Sreyash Pattnaik (24K81A05R7)** in fulfillment of the requirement for the award of degree of **BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING** is recorded of bonafide work carried out by them. The result embodied in this report have been verified and found satisfactory.

Signature of Guide
Mr. M. Dileep Kumar
Assistant Professor
Department of CSE

Signature of HOD
Mr. D. Krishna Kishore
Associate Professor & HOD
Department of CSE

Internal Examiner

External Examiner

St. MARTIN'S ENGINEERING COLLEGE

UGC AUTONOMOUS

Affiliated to JNTUH, Approved by AICTE

Accredited by NBA & NAAC A+, ISO 9001-2008 Certified.

Dhulapally, Secunderabad-500 100

www.smec.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I, the student of **"Bachelor of Technology in Department of Computer Science & Engineering"**, session: 2024 - 2028**, St. Martin's Engineering College**, Dhulapally, Kompally, Secunderabad, hereby declare that the work presented in this project work entitled **"JAVASCRIPT TESTING"** is the outcome of my own bonafide work and is correct to the best of my knowledge and this work has been undertaken taking care of Engineering Ethics. This result embodied in this project report has not been submitted in any university for award of any degree.

Sreyash Pattnaik (24K81A05R7)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowded our efforts with success.

First and foremost, I would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the facilities available in the Institute.

I especially would like to express our deep sense of gratitude and indebtedness to **Prof. Dr. K. Ravindra**, Director & Principal, St. Martin's Engineering College Dhulapally, for permitting us to undertake this project.

I am also thankful to **Mr. D. Krishna Kishore**, Head of the Department, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, Secunderabad, for his support and guidance throughout our project.

I would like to express my sincere gratitude and indebtedness to my project supervisor **Mr. M. Dileep Kumar**, Assistant Professor, Computer Science and Engineering, St. Martin's Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, I express thanks to all those who have helped us successfully completing this project. Furthermore, I would like to thank my family and friends for their moral support and encouragement. I express thanks to all those who have helped us in successfully completing the project.

Sreyash Pattnaik (24K81A05R7)

CONTENTS

1.ABSTRACT	**01**
2.INTRODUCTION	**02**
3.LITERATURE SURVEY	**03**
4.SYSTEM REQUIREMENTS	**04**
5.FLOW CHART	**05**
6.ALGORITHM	**06**
7.SOURCE CODE	**07**
8.EXPERIMENTAL RESULTS	**13**
9.CONCLUSION	**15**
10.FUTURE ENHANCEMENTS	**16**
11.REFERENCES	**17**

1. ABSTRACT

JavaScript testing is the practice of writing automated programs that verify whether JavaScript code behaves as expected. This project demonstrates how unit testing can be applied to a JavaScript application using Jest, a popular open-source testing framework that runs on Node.js. The project focuses on testing the ordering module of a food delivery app called QuickBite: adding and removing dishes, calculating the bill, applying delivery fees and coupon codes, validating inputs, and placing an order using mocked payment and delivery services. Each behaviour is described by a small test written in plain JavaScript, using functions such as describe, test, and expect. Running the test suite with a single command reports which tests pass or fail and how much of the source code is covered. The main objective of this project is to understand the working principles of JavaScript testing and to demonstrate how automated tests detect bugs early, prevent regressions, and improve the reliability and maintainability of software in a DevOps environment.

TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

2. INTRODUCTION

Software testing is an essential part of modern software development. In traditional development, programmers often check their code manually by running the application and trying a few inputs. When an application grows, this manual checking becomes time-consuming and unreliable, and a change in one part of the code may silently break another part. JavaScript, which powers websites, servers, and mobile applications, is no exception. Automated testing addresses this problem by allowing developers to describe the expected behaviour of the code once and re-run those checks whenever the code changes.

JavaScript tests are commonly grouped into unit tests, which check a single function or class; integration tests, which check how several modules work together; and end-to-end tests, which check the complete application through a browser. This project concentrates on unit testing with Jest. Jest provides a test runner, an assertion library, mock functions, and a code coverage report in a single package. In this project, the ordering module of a food delivery app is tested through a sequence of predefined test cases, including an order placement test that uses mocked payment and delivery services. The project illustrates the importance of automated testing and Continuous Integration in DevOps and shows how testing improves correctness, repeatability, and developer confidence.

UGC AUTONOMOUS

3.LITERATURE SURVEY

The growth of web applications, single-page applications, and continuous delivery has increased the need for automated quality assurance. Traditional testing methods depend heavily on manual checking, which becomes difficult to repeat consistently as the size of a code base grows. These limitations have encouraged the adoption of automated testing frameworks that support repeatable and standardized verification.

Testing frameworks such as Jest, Mocha, Jasmine, and Vitest help developers verify JavaScript code. Jest is particularly useful because it works with little configuration, includes built-in assertions, mocking, and coverage reports, and runs tests in parallel. Cypress and Playwright extend this idea to end-to-end browser testing, while Testing Library encourages testing user-visible behaviour of interface components.

Test-Driven Development (TDD) is another important concept in modern practice. In TDD, the developer writes a failing test first, then writes the code needed to pass it, and finally cleans up the code. Tests are stored in version control together with the source code and are run automatically in Continuous Integration pipelines.

The proposed project applies these concepts to a practical scenario. It demonstrates how Jest can execute predefined unit tests, use mock functions, and produce observable results. The project provides a basic understanding of automated testing and can serve as a foundation for more advanced work involving integration testing, end-to-end testing, and CI/CD pipelines.

4. SYSTEM REQUIREMENTS

4.1 Hardware requirements

- Processor: Intel Core i3 or equivalent.
- RAM: Minimum 4 GB; 8 GB recommended.
- Storage: At least 5 GB of available space.
- Display: Standard laptop or desktop monitor.
- Network: Internet connection for installing Node.js and downloading packages from npm.

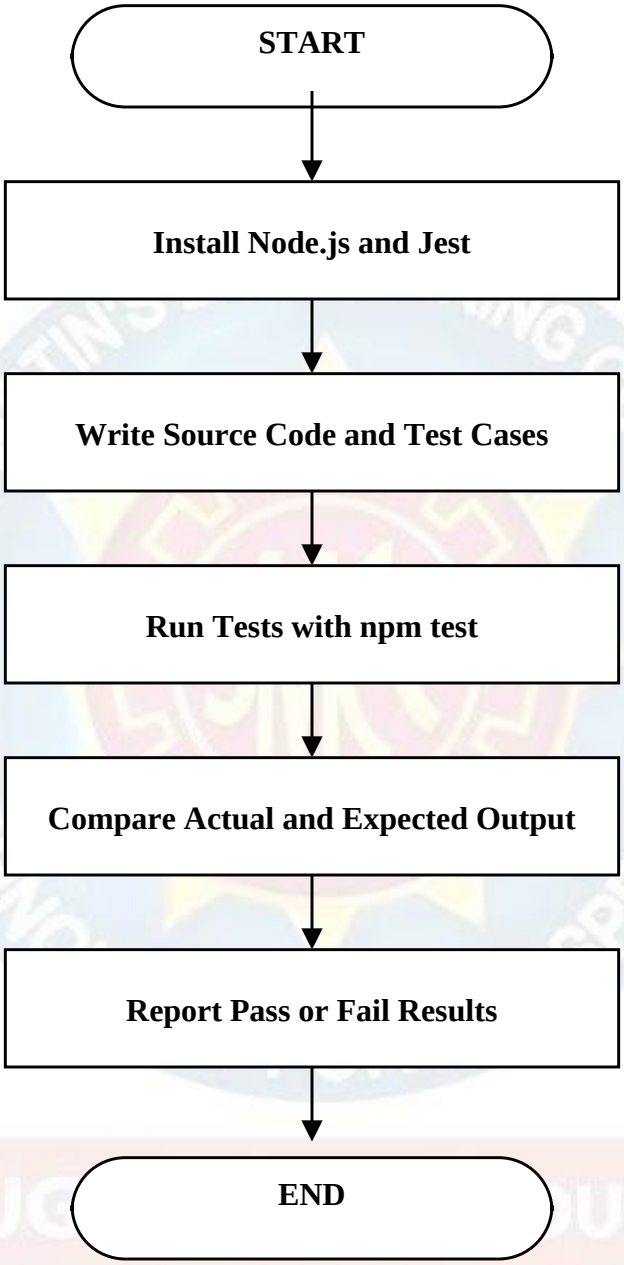
4.2 Software requirements

- Operating System: Windows 10/11, Ubuntu Linux, or macOS.
- Runtime: Node.js (current LTS version) with npm.
- Testing Framework: Jest.
- Programming Language: JavaScript (ES6+).
- Terminal: Command Prompt, PowerShell, or Linux Terminal.
- Code Editor: Visual Studio Code or any text editor.
- Package Manager: npm.
- Version Control: Git and GitHub, if required for storing the project files.

4.3 Functional requirements

- Implement an order module with add, remove, coupon, bill, delivery fee, and place-order functions.
- Write unit tests for every function using Jest.
- Test both valid inputs and invalid inputs (error handling).
- Use mock functions to simulate the payment and delivery services.
- Run all tests with one command and display the results.
- Generate a code coverage report.

5. FLOW CHART


START
Install Node.js and Jest
Write Source Code and Test Cases
Run Tests with npm test
Compare Actual and Expected Output
Report Pass or Fail Results
END

6. ALGORITHM

1.Start the process.

2.Install Node.js and npm on the system.

3.Create the project folder and initialize it with a package.json file.

4.Install Jest as a development dependency.

5.Write the source module (order.js) containing the functions to be tested.

6.Write the test file (order.test.js) with test cases for valid and invalid inputs.

7.Create mock payment and delivery services for the order placement test.

8.Run the test suite using the npm test command.

9.Jest loads each test file and executes every test case.

10. Compare the actual output of each function with the expected output.

11. Mark each test as passed or failed, and fix the code if any test fails.

12. Generate the code coverage report.

13. Display the test results and stop the process.

7. SOURCE CODE

Automated Unit Testing of a Food Delivery App Using Jest

1. Package File (package.json)

```
{
  "name": "quickbite-js-testing",
  "version": "1.0.0",
  "description": "JavaScript unit testing demo using Jest",
  "scripts": {
    "test": "jest --verbose --coverage"
  },
  "devDependencies": {
    "jest": "^29.7.0"
  }
}
```

2. Source Module (order.js)

```
const COUPONS = { WELCOME20: 20, SAVE10: 10 };

class Order {
  constructor(restaurant) {
    this.restaurant = restaurant;
    this.items = [];
    this.discount = 0; // percentage
  }

  addItem(name, price, quantity = 1) {
    if (typeof name !== "string" || name.trim() === "") {
      throw new Error("Dish name is required");
    }
    if (typeof price !== "number" || price <= 0) {
      throw new Error("Price must be a positive number");
    }
    if (!Number.isInteger(quantity) || quantity <= 0) {
      throw new Error("Quantity must be a positive integer");
    }
    const existing = this.items.find((item) => item.name === name);
    if (existing) {
      existing.quantity += quantity;
    } else {
```

```
      this.items.push({ name, price, quantity });
    }
  }

  removeItem(name) {
    const index = this.items.findIndex((item) => item.name === name);
    if (index === -1) {
      throw new Error("Dish not found in order");
    }
    this.items.splice(index, 1);
  }

  applyCoupon(code) {
    if (!(code in COUPONS)) {
      throw new Error("Invalid coupon code");
    }
    this.discount = COUPONS[code];
  }

  getSubtotal() {
    return this.items.reduce((sum, item) => sum + item.price * item.quantity, 0);
  }

  getDeliveryFee() {
    const subtotal = this.getSubtotal();
    if (subtotal === 0) return 0;
    return subtotal >= 500 ? 0 : 40;
  }

  getTotal() {
    const discounted = Math.round(this.getSubtotal() * (1 - this.discount / 100));
    return discounted + this.getDeliveryFee();
  }

  async placeOrder(paymentService, deliveryService) {
    if (this.items.length === 0) {
      throw new Error("Order is empty");
    }
    if (this.getSubtotal() < 100) {
      throw new Error("Minimum order value is Rs. 100");
    }
    const total = this.getTotal();
    const payment = await paymentService.charge(total);
    if (!payment.success) {
      return { status: "payment_failed" };
    }
```

```
        const delivery = await deliveryService.assignPartner(this.restaurant);
        this.items = [];
        this.discount = 0;
        return { status: "confirmed", total, partner: delivery.partnerName };
    }
}

module.exports = Order;
```

3. Test File (order.test.js)

```
const Order = require("./order");

describe("Order - adding dishes", () => {
    let order;
    beforeEach(() => {
        order = new Order("Spice Garden");
    });

    test("adds a new dish to the order", () => {
        order.addItem("Veg Biryani", 180);
        expect(order.items).toHaveLength(1);
        expect(order.items[0]).toEqual({ name: "Veg Biryani", price: 180, quantity: 1 });
    });

    test("increases quantity when the same dish is added again", () => {
        order.addItem("Masala Dosa", 90);
        order.addItem("Masala Dosa", 90, 2);
        expect(order.items).toHaveLength(1);
        expect(order.items[0].quantity).toBe(3);
    });

    test("rejects an empty dish name", () => {
        expect(() => order.addItem("", 100)).toThrow("Dish name is required");
    });

    test("rejects a negative price", () => {
        expect(() => order.addItem("Tea", -10)).toThrow("Price must be a positive number");
    });

    test("rejects a zero quantity", () => {
        expect(() => order.addItem("Tea", 20, 0)).toThrow("Quantity must be a positive integer");
    });
});

describe("Order - removing dishes", () => {
    test("removes an existing dish", () => {
```

```
    const order = new Order("Spice Garden");
    order.addItem("Masala Dosa", 90);
    order.removeItem("Masala Dosa");
    expect(order.items).toHaveLength(0);
  });

  test("throws when the dish is not in the order", () => {
    const order = new Order("Spice Garden");
    expect(() => order.removeItem("Pizza")).toThrow("Dish not found in order");
  });
});

describe("Order - bill calculation", () => {
  let order;
  beforeEach(() => {
    order = new Order("Spice Garden");
    order.addItem("Veg Biryani", 180, 2);
    order.addItem("Masala Dosa", 90);
  });

  test("calculates the subtotal", () => {
    expect(order.getSubtotal()).toBe(450);
  });

  test("charges a delivery fee of 40 below 500", () => {
    expect(order.getDeliveryFee()).toBe(40);
    expect(order.getTotal()).toBe(490);
  });

  test("gives free delivery on orders of 500 or more", () => {
    order.addItem("Paneer Butter Masala", 220);
    expect(order.getDeliveryFee()).toBe(0);
    expect(order.getTotal()).toBe(670);
  });

  test("applies the WELCOME20 coupon", () => {
    order.applyCoupon("WELCOME20");
    expect(order.getTotal()).toBe(400);
  });

  test("rejects an invalid coupon code", () => {
    expect(() => order.applyCoupon("FREEFOOD")).toThrow("Invalid coupon code");
  });

  test("returns 0 for an empty order", () => {
    expect(new Order("Spice Garden").getTotal()).toBe(0);
  });
```

```
});

describe("Order - placing an order with mocked services", () => {
  const makeOrder = () => {
    const order = new Order("Spice Garden");
    order.addItem("Veg Biryani", 180, 2);
    order.addItem("Masala Dosa", 90);
    return order;
  };

  test("charges the total, assigns a partner and confirms the order", async () => {
    const order = makeOrder();
    const payment = { charge: jest.fn().mockResolvedValue({ success: true }) };
    const delivery = { assignPartner: jest.fn().mockResolvedValue({ partnerName: "Ravi" }) };
    const result = await order.placeOrder(payment, delivery);
    expect(payment.charge).toHaveBeenCalledWith(490);
    expect(delivery.assignPartner).toHaveBeenCalledWith("Spice Garden");
    expect(result).toEqual({ status: "confirmed", total: 490, partner: "Ravi" });
    expect(order.items).toHaveLength(0);
  });

  test("does not assign a partner when payment fails", async () => {
    const order = makeOrder();
    const payment = { charge: jest.fn().mockResolvedValue({ success: false }) };
    const delivery = { assignPartner: jest.fn() };
    const result = await order.placeOrder(payment, delivery);
    expect(result.status).toBe("payment_failed");
    expect(delivery.assignPartner).not.toHaveBeenCalled();
  });

  test("throws when the order is empty", async () => {
    const order = new Order("Spice Garden");
    await expect(order.placeOrder({}, {})).rejects.toThrow("Order is empty");
  });

  test("throws when the order is below the minimum value", async () => {
    const order = new Order("Spice Garden");
    order.addItem("Tea", 40);
    await expect(order.placeOrder({}, {})).rejects.toThrow("Minimum order value is Rs. 100");
  });
});
```

4. Explanation of the Test Suite

1. Test grouping: describe() groups related tests, and test() defines one test case with a clear name.

2. Setup: beforeEach() creates a fresh Order before every test so that tests do not affect each other.

3. Assertions: expect(...) with matchers such as toBe, toEqual, toHaveLength, and toThrow compares the actual result with the expected result.

4. Error handling tests: Invalid names, prices, quantities, and coupon codes are tested using toThrow to confirm that the correct error message is raised.

5. Calculation tests: A subtotal of 450 plus a delivery fee of 40 gives 490, free delivery on orders of 500 or more is verified, and the 20 percent WELCOME20 coupon gives a total of 400.

6. Mock function: jest.fn().mockResolvedValue() replaces the real payment and delivery services, so order placement is tested without a network call, and toHaveBeenCalledWith confirms the amount charged and the restaurant sent to the delivery service.

7. Running the tests: The command npm test runs Jest with coverage and displays the pass or fail status of every test.

UGC AUTONOMOUS

8. EXPERIMENTAL RESULTS

The command **npm test** was used to run the test suite. Jest executed the 17 test cases in the file order.test.js and reported each one as passed. The output is shown below:

```
C:\Users\SREYASH\OneDrive\Desktop\js-testing-project\js-testing>npm test

> quickbite-js-testing@1.0.0 test
> jest --verbose --coverage

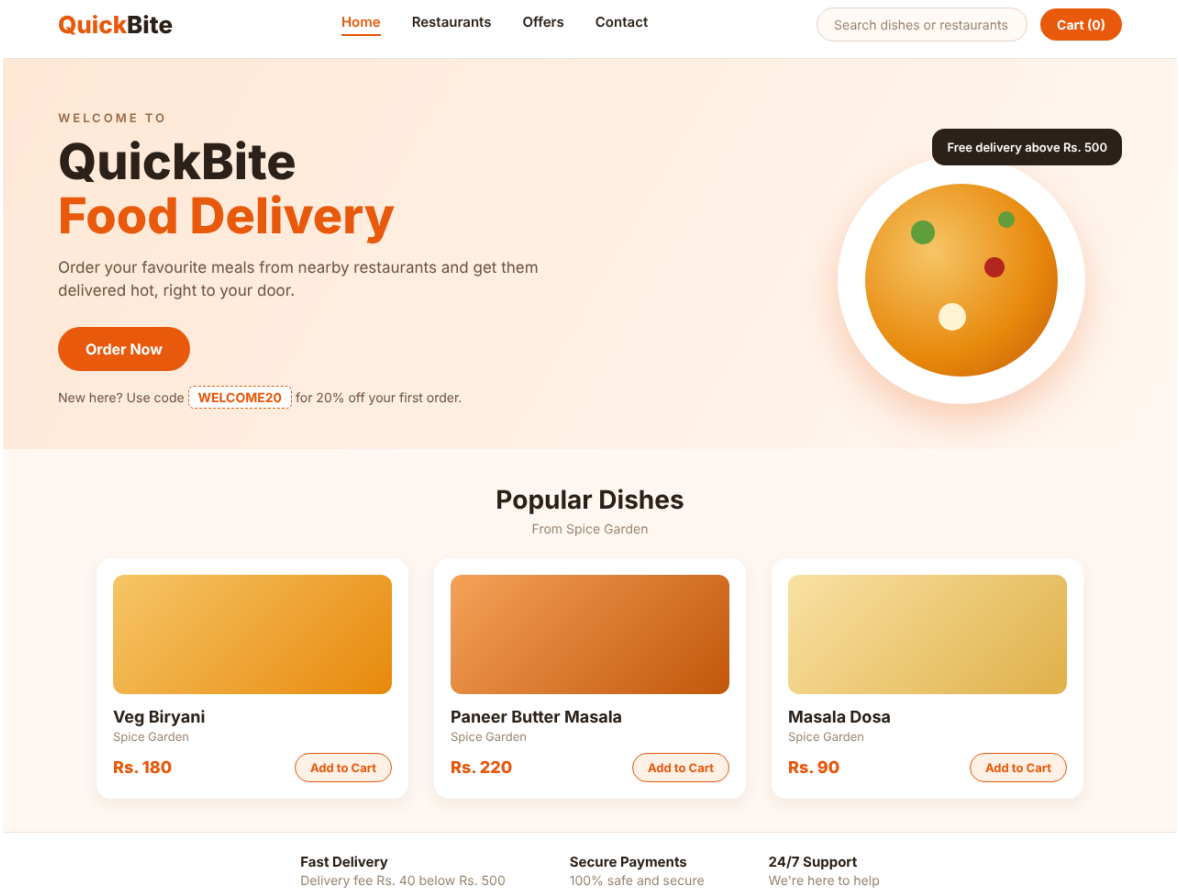
 PASS  ./order.test.js
  Order - adding dishes
    ✓ adds a new dish to the order (3 ms)
    ✓ increases quantity when the same dish is added again (1 ms)
    ✓ rejects an empty dish name (9 ms)
    ✓ rejects a negative price (1 ms)
    ✓ rejects a zero quantity (1 ms)
  Order - removing dishes
    ✓ removes an existing dish
    ✓ throws when the dish is not in the order (1 ms)
  Order - bill calculation
    ✓ calculates the subtotal
    ✓ charges a delivery fee of 40 below 500
    ✓ gives free delivery on orders of 500 or more (1 ms)
    ✓ applies the WELCOME20 coupon
    ✓ rejects an invalid coupon code
    ✓ returns 0 for an empty order
  Order - placing an order with mocked services
    ✓ charges the total, assigns a partner and confirms the order (1 ms)
    ✓ does not assign a partner when payment fails (1 ms)
    ✓ throws when the order is empty
    ✓ throws when the order is below the minimum value (1 ms)

----------|---------|----------|---------|---------|-------------------
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------|---------|----------|---------|---------|-------------------
All files |     100 |      100 |     100 |     100 |
 order.js |     100 |      100 |     100 |     100 |
----------|---------|----------|---------|---------|-------------------
Test Suites: 1 passed, 1 total
Tests:       17 passed, 17 total
Snapshots:   0 total
Time:        0.623 s
Ran all test suites.
```

Test group	Tests	Result
Adding dishes	5	Passed
Removing dishes	2	Passed
Bill calculation	6	Passed
Placing an order (mocked services)	4	Passed
Total	**17**	**17 passed, 0 failed**

Jest also reported 100% statement, branch, function, and line coverage for order.js, and the whole suite finished in 0.623 seconds.

The landing page of the QuickBite food delivery app, whose ordering logic was tested above, is shown below.



UGC AUTONOMOUS

9. CONCLUSION

The project “JavaScript Testing” demonstrates how automated unit testing verifies the correctness of a JavaScript application. Jest is used to test the ordering module of a food delivery app, covering adding and removing dishes, bill calculation, delivery fees, coupons, input validation, and order placement with mocked payment and delivery services. By using small, named test cases and a single test command, the verification process becomes systematic, repeatable, and fast.

This project helps detect bugs early, prevent regressions, save time, and improve confidence when code is changed. It also provides practical knowledge of software testing, mocking, DevOps concepts, and quality assurance. In the future, the project can be enhanced by adding integration and end-to-end tests, browser testing tools, and CI/CD pipelines using GitHub Actions or Jenkins to make the application more reliable, secure, and efficient.

ST. MARTIN'S ENGINEERING COLLEGE
SMEC
TECHNOLOGY FOR PROSPERITY
UGC AUTONOMOUS

10. FUTURE ENHANCEMENTS

1.Integration Testing: Test several modules together, such as the cart, the product catalogue, and a database.

2.End-to-End Testing: Use Cypress or Playwright to test complete user journeys in a real browser.

3.Component Testing: Test user interface components using Testing Library.

4.Test-Driven Development: Write tests before the code for every new feature.

5.Code Coverage Targets: Set minimum coverage thresholds and fail the build when they are not met.

6.CI/CD Integration: Run the tests automatically using GitHub Actions or Jenkins on every code change.

7.Performance and Security Testing: Add load testing, accessibility checks, and dependency vulnerability scans.

11. REFERENCES.

1.Jest Documentation. Getting Started. https://jestjs.io/docs/getting-started

2.Jest Documentation. Using Matchers. https://jestjs.io/docs/using-matchers

3.Jest Documentation. Mock Functions. https://jestjs.io/docs/mock-functions

4.Node.js Documentation. https://nodejs.org/en/docs

5.MDN Web Docs. JavaScript Guide. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

6.Jenkins Documentation. Jenkins User Documentation. https://www.jenkins.io/doc/